

Maya Siddiqui-Patel

Dr. Xihao Xie

CS 5393

04/10/2025

Exam 2: Exploring Open-Source LLMs with Ollama

Github Repository: <https://github.com/msiddiqui-patel/CS5393Exam2.git>

Introduction

Open-source large language models (LLMs) have rapidly transformed natural language processing (NLP) by offering powerful capabilities without the high costs of proprietary systems. Over the course of this project, I have explored these models for tasks such as text generation, summarization, question answering, code completion, and multilingual translation. This report presents a comprehensive evaluation of three open-source LLMs—Phi, Mistral, and Deepseek-R1:14b—by examining their speeds, outputs, resource usage, and limitations in both English-language benchmarks and multilingual tasks.

The document is structured into six key sections. Section (a) explains why these three models were chosen and how each addresses particular use-case needs. Section (b) describes the detailed results from general exploration (question answering, summarization, code generation, and creative writing), and focused experimentation of multilingual evaluations. Section (c), provides an analysis of each model’s strengths and weaknesses, highlighting trade-offs between speed, resource usage, and output depth. Section (d) provides broader insights into the growing open-source LLM ecosystem-- especially considering reasoning transparency and multilingual capabilities. Section (e) reflects on the practical real-world implications of these findings, emphasizing the importance of matching models to specific deployment scenarios. Finally,

Section (f) covers the main challenges I faced during the assignment and explains how I resolved them.

(a) Model Selection Overview

My goal in choosing these models was to capture different points along the spectrum of performance, scale, and specialization:

- Phi: developed with a strong emphasis on speed and low-resource usage, Phi stands out for how efficiently it can respond in real-time contexts. The model is relatively small compared to the others, yet it still offers a workable degree of coherence on standard tasks.
- Mistral: at around 7B parameters, Mistral provides a middle ground, offering balanced performance in reasoning and multilingual tasks. I selected it to see whether a moderately sized model can provide a strong trade-off between quality and hardware demands.
- Deepseek-R1:14b: with 14B parameters, Deepseek is the largest model in this study. It promises more elaborate reasoning and chain-of-thought transparency. I included it to assess how significantly a larger model's detail and depth might outweigh its resource costs and slower response times.

By looking at how these three models perform, I hoped to understand how one might balance speed and efficiency (Phi), a versatile middle ground (Mistral), and the potential for deeper, more reflective outputs (Deepseek).

(b) Detailed Results from Basic Exploration and Focused Experimentation

Experimental Setup

I ran two main sets of tests. The first set focused on standard tasks:

1. General Question Answering (QA): "What are black holes and how are they formed ?"

2. Summarization: using a paragraph about Athena Parthenos and its historical background.
3. Code Generation: prompting each model to create a simple Python function to compute an average.
4. Creative Writing: asking them to produce a short story opening about a hidden door leading to a realm where memories serve as currency.

The second set consisted of multilingual tasks, testing French, Spanish, and Chinese prompts, as well as bidirectional English-French translation. To ensure that cold-start overhead did not produce false timing, I added a warm-up routine to load the models beforehand. Each task was run multiple times, with results averaged to offset minor fluctuations caused by system load or memory management.

Standard Benchmark Results

Phi's Performance

Phi consistently delivered the fastest responses across all four standard tasks. On general QA, it provided concise, factually decent accounts of black holes, referencing gravitational collapse without overelaborating. This concise style also appeared in summarization—Phi neatly extracted the core facts about Pheidias, the gold and ivory materials, and the statue's historical context-- though it glossed over deeper cultural angles.

In code generation, Phi produced a working Python snippet for calculating an average. While it nailed the fundamentals, it did not include any advanced error handling or extensive comments. The fast response time (often well under 3 seconds), remained its greatest advantage. However, in creative writing, Phi's outputs sometimes felt abrupt. For instance, when describing a hidden door that leads to a world where memories function as currency, it ended the narrative quickly, suggesting limited capacity for extended creativity.

Memory usage was minimal, fluctuating at around ± 0.5 MB in many runs. For applications like real-time chatbots, or systems running on lower-power devices, Phi's efficiency can be a major asset.

Mistral's Performance

Mistral presented a more balanced profile in both speed and output richness. Typical inference times ranged from about 3 to 6 seconds, slower than Phi but nowhere near as slow as Deepseek. On the summarization prompt about Athena Parthenos, Mistral displayed a fair bit of nuance, not only mentioning the materials used in the statue, but also the cultural prominence of Pheidias as a renowned Greek sculptor.

In code generation, Mistral often returned the requested Python function with basic error checks or comments, making it more user-friendly for those wanting to understand how the code worked. For general QA, it gave slightly more detailed explanations, referencing event horizons and gravity wells when asked about black holes. This additional detail might be beneficial for users who prefer context, though it also meant a modest rise in inference time.

When it came to creative writing, Mistral produced well-structured mini-narratives. On the memories-as-currency prompt, it added light but evocative details about how those memories might be stored or traded. Although not as lavishly descriptive as Deepseek, it went well beyond Phi's straightforward approach. Memory usage for Mistral hovered around ± 0.4 MB, making it feasible for mid-range hardware setups where concurrency or slightly more detailed outputs are desirable.

Deepseek-R1:14b's Performance

As expected for a 14B-parameter model, Deepseek was the slowest—sometimes taking more than 16 seconds for short prompts and exceeding 60 seconds for tasks it deemed complex.

However, the model offered a depth and nuance absent in the other two models. In summarization, it didn't just distill the main points about Athena Parthenos but also considered historical context, referencing how ancient Greek artistry evolved and subtly discussing the concept of "lost wonders."

Deepseek also frequently inserted chain-of-thought tokens, like `<think>` and `</think>`, which provided an introspective glimpse into its reasoning steps. This could be invaluable in debugging or specialized research contexts but might feel excessive or slow in production settings. In code generation, Deepseek's solution for a Python average function not only worked but also included disclaimers, docstrings, and alternative error-handling strategies, underscoring the model's thoroughness.

For creative writing, Deepseek created atmospheres brimming with detail—using imagery and evocative language to craft an immersive scenario. That immersive style can be beneficial in certain use cases, such as narrative design or drafting longer-form content. However, memory overhead tended to be much higher, and a single prompt might consume enough resources to cause delays if multiple users were accessing the system concurrently.

Multilingual Tasks

The multilingual tests measured each model's ability to handle French, Spanish, Chinese, and translations:

- Phi struggled with languages beyond English, showing limited adaptability. For example, French outputs contained some relevant phrases but veered into non-sequiturs, and Spanish prompts lacked cultural depth or clarity.

- Mistral handled non-English text comparatively well. Its Spanish answers about “la vida” contained thoughtful references to cultural notions of life’s value, and translations between English and French were mostly accurate.
- Deepseek delivered the richest multilingual outputs, sometimes reflecting on the cultural or philosophical dimension of each language’s question. However, it could be extremely slow for translations, as it might analyze each lexical element in excessive detail.

Overall, Mistral’s multilingual balance stood out as the most practical option for applications needing coverage across several languages without incurring huge latency.

(c) Analysis of Strengths and Weaknesses

Phi

Strengths: Low latency, minimal resource usage, and straightforward text generation make Phi ideal for real-time or resource-constrained scenarios.

Weaknesses: Its brevity and limited multilingual capacity are clear drawbacks. Creative or complex tasks often yield underwhelming or incomplete content.

Mistral

Strengths: Balanced approach to speed, memory usage, and depth. Multilingual tasks are handled respectably, with translations generally accurate and culturally sensitive.

Weaknesses: Mistral is not as fast as Phi nor as detailed as Deepseek. It also might not scale to extremely large queries if a user needs chain-of-thought reasoning or extensive details.

Deepseek-R1:14b

Strengths: Offers thorough, methodical answers, along with a transparent reasoning style helpful for specialized domains such as legal or academic research. Creates some of the richest creative writing among the models.

Weaknesses: Slow inference time, substantial resource demands, and verbose chain-of-thought expansions that can be impractical for real-time or large-scale deployments.

(d) Insights Gained About Open-Source LLMs

My experiments yielded several broad takeaways regarding the current state of open-source LLM development:

1. Trade-offs are unavoidable: smaller models shine in speed and efficiency, but risk superficial results. Larger models go deeper but demand more time and memory.
2. Multilingual support differs sharply: diverse training data is critical. Mistral showed a stable cross-language approach, whereas Phi failed, likely due to less comprehensive multilingual exposure.
3. Reasoning transparency vs. practicality: Deepseek’s chain-of-thought tokens illustrate a path toward interpretability, yet this can be burdensome in operational contexts. There is an inherent negative correlation between clarity of process and efficiency of delivery.
4. Open-source ecosystems are advancing rapidly: models are constantly improved by the broader community. Regular updates and fine-tuning strategies open the door to better performance without proprietary lock-in.

(e) Reflections on Practical Implications for Real-World Applications

Matching a model’s capabilities to the deployment environment is paramount.

1. Latency-sensitive use cases: if the system is a chatbot or mobile-edge application where every second counts, Phi stands out as the best choice thanks to its minimal overhead. Even though it may produce less “imaginative” responses, sometimes that trade-off is acceptable to maintain a snappy user experience.

2. Multi-language educational or support platforms: Mistral’s balanced performance suggests its efficiency in building a platform that fields user queries from multiple regions—Mistral can ensure a moderate degree of detailed, accurate text across languages. Its moderate speed is generally tolerable for helpdesk interactions.
3. Thorough research and creative writing: Deepseek’s elaborate responses can be invaluable in academic or professional contexts needing both nuance and clarity about how a conclusion was reached. The waiting time of tens of seconds might be acceptable if the result is a more exhaustive exploration of a topic. For applications that rely on precise or justifiable reasoning—like a legal drafting helper—Deepseek’s chain-of-thought approach could be a key differentiator.
4. Potential hybrid strategies: a practical real-world system could route straightforward queries to Phi or Mistral while reserving Deepseek for specialized or high-stakes tasks. Such orchestration might balance user experience (minimizing wait times) with the benefits of a more advanced reasoning engine where needed.

From a cost perspective, running large models also entails heavier memory usage and potentially more expensive GPUs to support high concurrency. The overhead must be justified by a tangible improvement in output. In many workflows, a mid-range model like Mistral strikes the best equilibrium.

(f) Discussion of Challenges Faced and How I Overcame Them

This assignment required integrating multiple open-source LLMs under a unified benchmark.

Although promising, it was not without hurdles:

1. Ollama subprocess delays: running large prompts through Ollama sometimes gave the impression of a system “hang,” particularly with Deepseek. However, this result was

simply the model's extensive reasoning. I introduced timeout wrappers and made sure to separate warm-up times from benchmark times. This clarified what was truly latency versus what was loading overhead.

2. **Memory tracking inconsistencies:** psutil-based measurements occasionally spiked unpredictably due to other background processes on my machine. To address these inconsistencies, I conducted multiple runs of each task and averaged memory usage. While not perfect, this method yielded a more representative snapshot of resource consumption patterns for each model.
3. **Multilingual misalignment:** Phi's inconsistent results in French and Chinese prompted me to add short instructions in the target language, such as "répondez strictement en français" (strictly respond in French). Doing so sometimes helped keep the model on-topic. Nonetheless, the need for the phrase highlighted the need for better multilingual data or fine-tuning if a single model must excel across multiple languages.
4. **Overly verbose output:** Deepseek's chain-of-thought tokens made its logs difficult to inspect. I resolved this by storing a truncated version of the output in the main console logs (to maintain readability), while saving the full, unabridged responses to a secondary file if deeper analysis was ever needed.

Overall, these challenges underscored that open-source LLMs, while powerful, can be tricky to harness optimally. Each environment and project context will likely require adjusting timeouts, memory logging, or prompt engineering to achieve stable and reliable results.

Conclusion

In this individual project, I investigated three open-source LLMs --Phi, Mistral, and Deepseek-R1:14b-- to compare how they handle standard tasks and multilingual requests

under real-world constraints. Phi stands out for its exceptional speed and resource efficiency, Mistral delivers well-rounded performance across languages and tasks, and Deepseek impresses with thorough, interpretable outputs (but at the cost of heavier resource demands and longer wait times).

These findings emphasize that no single model can fulfill every use-case requirement. Instead, selecting or orchestrating multiple models might be the most effective strategy. For high-volume, quick-response scenarios, Phi's minimal latency is unbeatable. Where moderate detail and multilingual competence are essential, Mistral excels. In specialized or detail-oriented domains demanding extended reasoning, Deepseek's chain-of-thought approach can be uniquely valuable.

From a broader perspective, this assignment has highlighted how swiftly the open-source LLM community is evolving. As models keep improving, future iterations might blur the lines between speed and depth. Nevertheless, at present, trade-offs remain central. By matching model capabilities to the demands of each deployment environment, I can harness the potential of these powerful open-source tools while mitigating drawbacks like slow inference or high memory usage. Ultimately, the key lies in carefully aligning system objectives, user expectations, and each model's inherent strengths; this alignment process, while sometimes challenging, offers the freedom to tailor robust AI solutions without the constraints of proprietary ecosystems.

Works Cited

“deepseek-r1:14b.” *Ollama Library*, <https://ollama.com/library/deepseek-r1:14b>. Last Accessed 10 Apr. 2025.

“mistral.” *Ollama Library*, <https://ollama.com/library/mistral>. Last Accessed 10 Apr. 2025.

Ollama. <https://ollama.ai/download>. Last Accessed 10 Apr. 2025.

“Parthenon Athens Survival Intolerance Greed Bombs History.” *Lost Treasures Intolerance Greed*, <https://lost-treasures-intolerance-greed.com/parthenon-athens-survival-intolerance-greed-bombs-history.html>. Last Accessed 10 Apr. 2025.

“phi.” *Ollama Library*, <https://ollama.com/library/phi>. Last Accessed 10 Apr. 2025.